15 CHAPTERS COMPLETE GUIDE

AI & ML MASTERY



CONCEPTS • ALGORITHMS • INTERVIEWS • REAL WORLD

FROM FUNDAMENTALS TO REAL WORLD AI SYSTEMS

01 AI, ML & DL Overview

02 Problem Solving & Search

03 Knowledge Representation

04 Logic & Inference



05 Probability & Bayesian Reasoning

06 Statistics & Mathematics

07 Knowledge Representation

08 Logic & Inference

09 Probability & Bayesian Reasoning

10 Machine Learning Fundamentals

11 Supervised & Unsupervised Learning

12 Neural Networks & Deep Learning

13 NLP & Computer Vision



14 Generative AI, LLMs & RAG

15 AI Interview & Real World Concepts

CLEAR CONCEPTS
Easy explanations with diagrams & examples

INTERVIEW READY
Important questions & answers to crack any AI/ML interview

REAL WORLD FOCUS
Use cases, projects & industry applications in every topic

LEARN • BUILD • SOLVE
Build strong foundation, solve problems & create intelligent systems

MASTER AI TODAY, SHAPE TOMORROW.

Think Smarter | Build Better | Solve Bigger | Impact Future

1. AI FUNDAMENTALS

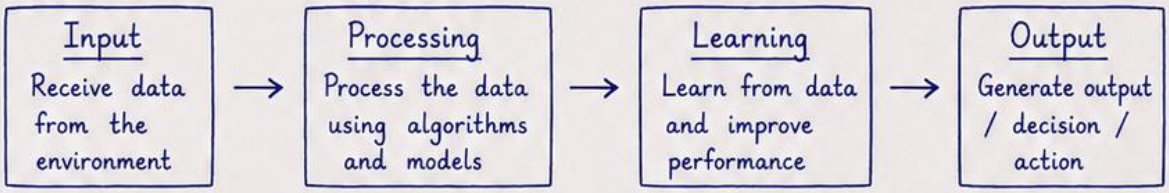
- **What is AI?**

  AI (Artificial Intelligence) is the simulation of human intelligence in machines that are programmed to think, learn and solve problems like humans.

- **Goals of AI**
  - To create systems that can think rationally
  - To create systems that can act intelligently
  - To create systems that can learn and adapt
  - To solve real-world problems efficiently

- **Types of AI**
  - Narrow AI (Weak AI) – Designed for a specific task
    e.g., Siri, Recommendation Systems
  - General AI (Strong AI) – Can perform any intellectual task that a human can do (Still a concept)
  - Super AI – AI that surpasses human intelligence in every aspect (Future concept)

- **Components of AI System**



- **Applications of AI**
  - Healthcare – Disease prediction, Drug discovery
  - Finance – Fraud detection, Algorithmic trading
  - Education – Personalized learning, Smart tutoring
  - Transport – Self-driving cars, Route optimization
  - Entertainment – Recommendations, Content generation

- **Key Terms**
  - Data – Raw facts and figures
  - Algorithm – Step-by-step instructions to solve a problem
  - Model – Trained system that makes predictions or decisions
  - Training – Feeding data to a model to learn patterns
  - Inference – Using a trained model to make predictions

Note :
AI is not about replacing humans, it's about augmenting human capabilities.

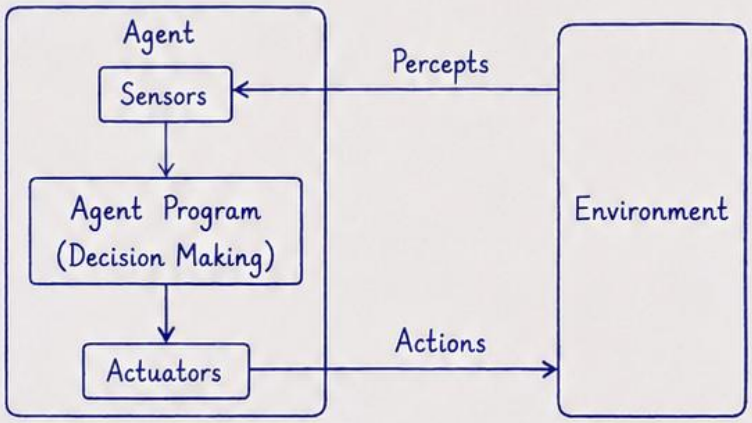
2. INTELLIGENT AGENTS

• What is an Intelligent Agent?

An intelligent agent is an entity that perceives its environment through sensors and acts upon that environment through actuators to achieve goals.

> **Key Idea :**
> Perceive → Decide → Act to achieve goals.

• Agent Architecture



Sensors : Receive information (percepts) from environment.

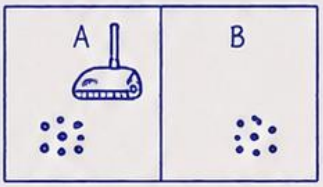
Actuators : Perform actions in the environment.

Agent Program : Maps percept sequences to actions.

• Types of Agents

① Simple Reflex Agent — Acts only on the basis of current percept.
② Model-Based Reflex Agent — Maintains an internal state (model) of the world.
③ Goal-Based Agent — Takes goals into account to decide actions.
④ Utility-Based Agent — Chooses actions that maximize expected utility.
⑤ Learning Agent — Improves performance from experience.

• Example : Vacuum Cleaner World



Percepts : [Location, Dirt?]
Actions : Left, Right, Suck, NoOp
Goal : Clean all locations.

> **Note :**
> An agent is intelligent if it does the right thing at the right time.

• PEAS Description of an Agent

P	Performance Measure — How well the agent does its job.
E	Environment — Everything outside the agent.
A	Actuators — How the agent affects the environment.
S	Sensors — How the agent perceives the environment.

→ Used to design good intelligent agents.

3. PROBLEM SOLVING & SEARCH

• What is Problem Solving?

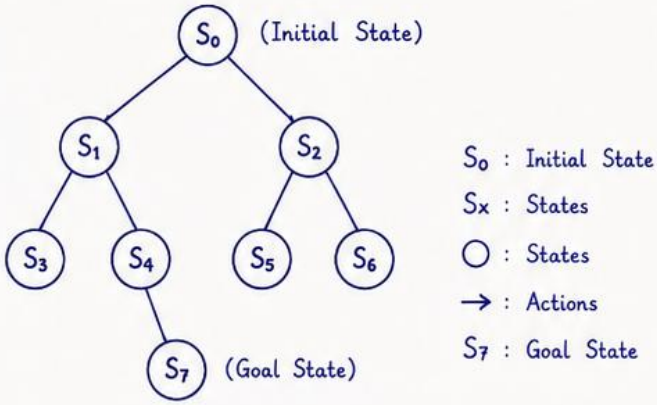
Problem solving is the process of finding a sequence of actions that transforms the initial state into a goal state by applying rules and operators.

> **Key Idea :**
> Search is the systematic exploration of possible solutions to find a goal.

• Problem Solving Formulation

Initial State	The starting point of the problem.
State Space	Set of all possible states.
Actions / Operators	Set of actions applicable in a state.
Transition Model	Resulting state after applying an action.
Goal Test	Condition that checks if goal is achieved.
Path Cost	Cost of a path (sum of step costs).
Solution	A sequence of actions that leads from initial state to goal state.

State Space Tree



• Types of Problems

① Single State Problems – Only one state is involved.
② Multiple State Problems – Many states may be involved.
③ Contingency Problems – Outcome is uncertain.
④ Exploration Problems – Information about the environment is incomplete.
⑤ Adversarial Problems – An opponent tries to achieve a goal opposite to the agent.

> **Example :**
> Route finding in a map,
> 8-puzzle problem,
> Vacuum cleaner world,
> Water jug problem.

• Basic Search Strategies

Search Strategy	Description	Complete	Optimal	Time Complexity	Space Complexity
1. Uninformed Search (Blind Search)	No knowledge about goal location.	Sometimes	Sometimes	High	High
2. Informed Search (Heuristic Search)	Uses heuristic (h) to guide search.	Sometimes	Sometimes	Medium	Medium
3. Heuristic Search	Uses evaluation function $f(n) = g(n) + h(n)$.	Sometimes	Sometimes	Medium to High	Medium to High
4. Optimal Search	Finds least cost solution.	Yes	Yes	High	High

• Heuristic Function

A heuristic function $h(n)$ estimates the cost from the current state n to the goal state.
Good heuristic ⇒ Faster search.
Examples: Manhattan distance, Euclidean distance.

• Evaluation Function

$f(n) = g(n) + h(n)$
Where,
$g(n)$ = cost from start to n
$h(n)$ = estimated cost from n to goal

4. BFS, DFS & UNIFORM COST SEARCH

codedsoul_05

- **Why these searches?**
  These are fundamental uninformed search algorithms used to explore the state space when no domain-specific knowledge (heuristic) is available.

Key Idea :
Explore the search space in different ways to find the goal state.

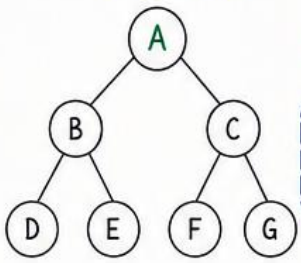
1. Breadth First Search (BFS)

- Explores all nodes level by level (left to right).
- Uses a FIFO queue.

Algorithm (BFS)

① Add initial state to the queue.
② Repeat until queue is empty :
- Remove the front node.
- If it is goal, return path.
- Add all unvisited successor nodes to the back of the queue.

Example Graph



BFS Traversal (starting from A)

A → B → C →
D → E → F → G

BFS Properties

✓ Complete? Yes
✓ Optimal? Yes (for unit costs)
✓ Time Complexity : $O(b^d)$
✓ Space Complexity : $O(b^d)$

Where,
b = branching factor
d = depth of shallowest goal

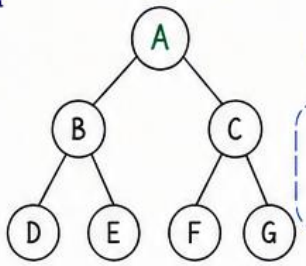
2. Depth First Search (DFS)

- Explores as far as possible along each branch before backtracking.
- Uses a LIFO stack or recursion.

Algorithm (DFS)

① Add initial state to the stack.
② Repeat until stack is empty :
- Pop the top node.
- If it is goal, return path.
- Push all unvisited successors onto the stack.

Example Graph



DFS Traversal (starting from A)

A → B → D →
E → C → F → G

DFS Properties

✓ Complete? No (can get stuck in infinite path)
✓ Optimal? No
✓ Time Complexity : $O(b^m)$
✓ Space Complexity : $O(bm)$

Where,
b = branching factor
m = maximum depth of the tree

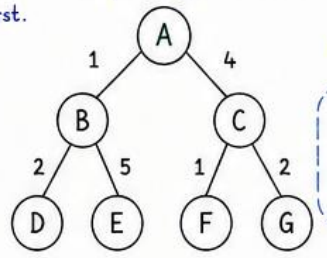
3. Uniform Cost Search (UCS)

- Explores node with the lowest path cost first.
- Uses a priority queue (min-heap).

Algorithm (UCS)

① Add initial state to priority queue with cost = 0.
② Repeat until queue is empty :
- Remove node with lowest cost.
- If it is goal, return path.
- Add all successors with updated path cost to the queue.

Example Graph with Costs



UCS Traversal (starting from A)

A → B → D →
F → G → E → C

UCS Properties

✓ Complete? Yes
✓ Optimal? Yes (for any positive costs)
✓ Time Complexity : $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
✓ Space Complexity : $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

Where,
C^* = cost of optimal solution
ε = minimum step cost

- **Comparison Summary**

Algorithm	Data Structure	Explores	Complete	Optimal	Time Complexity	Space Complexity	Best Used When
BFS	Queue (FIFO)	Level by level	Yes	Yes (unit costs)	$O(b^d)$	$O(b^d)$	Shallowest solution needed
DFS	Stack (LIFO)	Deep as possible	No	No	$O(b^m)$	$O(bm)$	Memory is limited, solution may be deep
UCS	Priority Queue (Min-Heap)	Lowest cost first	Yes	Yes (positive costs)	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	Varying step costs, optimal path needed

- **Notes**
  ✓ BFS is a special case of UCS when all step costs are equal.
  ✓ DFS uses less memory but may not find the shortest or any solution.
  ✓ UCS guarantees the least-cost solution when step costs are positive.

5. HEURISTIC SEARCH & A*

codedsoul_05

What is Heuristic Search?

Heuristic search uses a heuristic function (h) to estimate the cost from the current state to the goal. It guides the search in the right direction and reduces the search space.

Key Idea :

Use knowledge (experience or evidence) to make an informed guess (heuristic) about which path is likely to reach the goal cheapest.

Heuristic Function h(n)

Estimates the cost from node n to the goal.

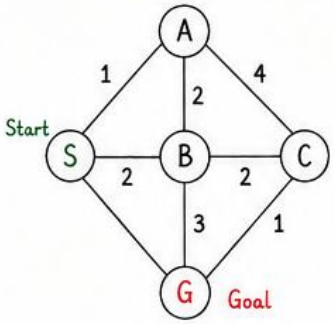
Examples :

- Manhattan Distance
- Euclidean Distance
- Straight Line Distance

Properties of a Good Heuristic

- ✓ Admissible : h(n) never overestimates the true cost. $h(n) \leq h^*(n)$
- ✓ Consistent (Monotonic) : $h(n) \leq c(n,n') + h(n')$ for every edge (n, n').
- ✓ Informed : Provides useful guidance (closer to true cost).
- ✓ Efficient : Reduces time and space by exploring fewer nodes.

Example Graph



Numbers on edges are step costs.
h(n) is estimated cost to Goal (G).

Heuristic h(n)

h(S) = 5
h(A) = 4
h(B) = 2
h(C) = 1
h(G) = 0

Common Heuristics

- Manhattan Distance (Grid)
  $h(n) = |x_n - x_g| + |y_n - y_g|$
- Euclidean Distance
  $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$
- Straight Line Distance
  h(n) = direct distance between n and goal

A* Search Algorithm

A* (A-star) combines the actual cost from start to current node and the heuristic cost to the goal.

$$f(n) = g(n) + h(n)$$

- g(n): Actual cost from start to n
- h(n): Heuristic cost from n to goal
- f(n): Estimated total cost of best path through n

A* Example on the Graph

Path	g(n)	h(n)	f(n) = g(n)+h(n)
S → A	1	4	5
S → B	2	2	4
S → A → C	1 + 4 = 5	1	6
S → B → C	2 + 2 = 4	1	5
S → B → G	2 + 3 = 5	0	5
S → A → C → G	1+4+1=6	0	6

Steps of A*

1. Add start node S to open list with g = 0.
2. Pick node with lowest f(n) from open list.
3. If goal reached, stop and return path.
4. Otherwise, expand the node and update neighbour nodes.
5. Repeat until goal is found or open list is empty.

Comparison : BFS / DFS / UCS / A*

Algorithm	Strategy	Uses Heuristic?	Optimal?	Complete?	Time Complexity	Space Complexity	Best Used When
BFS	Level by level	No	Yes (unit cost)	Yes	$O(b^d)$	$O(b^d)$	Shallowest solution or least steps (unit cost)
DFS	Deep as possible	No	No	No	$O(b^m)$	$O(bm)$	Memory is limited
UCS	Lowest cost first	No	Yes	Yes	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	Varying step costs, optimal solution
A*	Best estimated total cost first	Yes	Yes (if h is admissible)	Yes	Exponential (worst case)	Exponential (worst case)	Large search space with good heuristic

Notes

- ✓ If h(n) = 0 for all nodes, A* becomes Uniform Cost Search.
- ✓ If all step costs are 1 and h(n) = 0, A* becomes BFS.
- ✓ The better the heuristic, the fewer nodes A* explores.
- ✓ Admissible & consistent heuristic guarantees optimal solution.

When to Use A*?

- When the search space is large.
- When we have a good heuristic function.
- When optimal solution is required.
- Common in path finding, game AI, robotics, GPS navigation.

6. GAME PLAYING & MINIMAX

codedsoul_05

What is Game Playing?

Game playing is a way of decision making in situations of conflict where outcomes depend on the actions of multiple agents (opponents). The goal is to choose the best move to win or achieve the best possible outcome.

Key Idea :

Assume the opponent also plays optimally. Choose the move that maximizes our minimum possible gain.

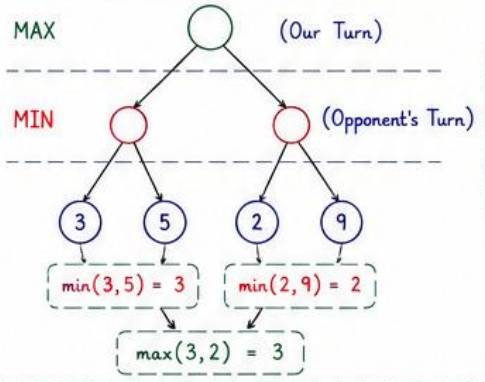
Examples of Games :

- Tic-Tac-Toe
- Chess
- Checkers
- Othello
- Connect Four

Components of a Game

- ✓ States : All possible positions in the game.
- ✓ Initial State : The starting position.
- ✓ Players : Typically MAX (us) and MIN (opponent).
- ✓ Actions : Legal moves available to a player.
- ✓ Transition Model : Resulting state after a move.
- ✓ Terminal State : No more moves possible.
- ✓ Utility Function : Numeric value of terminal states (win = +1, lose = -1, draw = 0).

Example: Simple Game Tree



- Green (MAX) : Our moves
- Red (MIN) : Opponent's moves
- Blue (Numbers) : Utility values of terminal states

Best move for MAX is the left branch (value = 3).

Minimax Algorithm

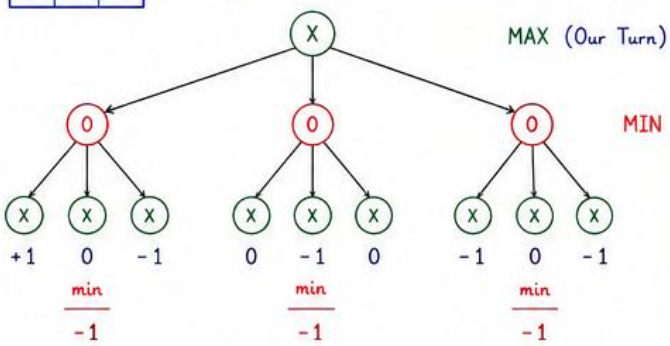
Minimax is a recursive algorithm that chooses the move that leads to the best outcome assuming the opponent also plays optimally.

```
Minimax(state, depth, isMax)
1. If state is terminal or depth = 0
      return Utility(state)
2. If isMax (our turn):
      best = -∞
      For each action in Actions(state):
         value = Minimax(Result(state, action), depth-1, False)
         best = max(best, value)
      return best
3. Else (opponent's turn):
      best = +∞
      For each action in Actions(state):
         value = Minimax(Result(state, action), depth-1, True)
         best = min(best, value)
      return best
```

Example : Tic-Tac-Toe (One Step Lookahed)

X		O
	X	

X is MAX (Our turn)
O is MIN (Opponent)
Utility: X win = +1, Draw = 0, O win = -1



MAX chooses any move (all have value -1 in this example)

Summary of Game Playing Approaches

Approach	Knowledge Required	Guarantees	Time Complexity	Space Complexity	Best Used When
Minimax	Rules of the game, Utility of terminal states	Optimal (if full tree is explored)	$O(b^m)$	$O(bm)$	Game tree is small enough to explore completely
Minimax + Depth Limit	Rules, Heuristic Evaluation Function	Good (not optimal)	$O(b^m)$ (m = depth limit)	$O(bm)$	Large game trees, use heuristic to evaluate leaf nodes
Alpha-Beta Pruning (Improved Minimax)	Same as Minimax	Optimal (if same depth explored)	$O(b^{m/2})$ (best case)	$O(bm)$	Large game trees, reduces unnecessary branches

Notes

- ✓ b = branching factor (average number of children per node)
- ✓ m = maximum depth of the game tree
- ✓ Utility function is applied only at terminal states (or at depth limit in heuristic search).
- ✓ Alpha-Beta pruning skips branches that cannot affect the final decision.

Where is it Used?

- Chess engines (Stockfish, AlphaZero)
- Checkers, Othello, Connect Four
- Decision making in adversarial situations
- AI in strategy games

7. KNOWLEDGE REPRESENTATION

codedsoul_05

What is Knowledge Representation?

Knowledge Representation (KR) is the way of structuring and encoding information about the world so that a computer can understand, reason and make decisions.

Key Idea :

Good representation makes it easy to store knowledge and perform reasoning efficiently.

Why is it Important?

- ✓ Helps in reasoning and inference
- ✓ Enables knowledge sharing
- ✓ Supports problem solving
- ✓ Foundation for intelligent systems

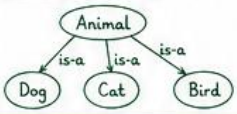
Properties of a Good Knowledge Representation

- ✓ Representational Adequacy : Can represent all necessary knowledge.
- ✓ Inferential Adequacy : Supports deriving new knowledge.
- ✓ Inferential Efficiency : Enables fast reasoning.
- ✓ Acquisitional Efficiency : Easy to add new knowledge.
- ✓ Clarity : Easy to understand and interpret.

Levels of Knowledge

Level	Description	Example
1. Conceptual Level	Represents general knowledge about objects, classes, relations.	Dog is an Animal.
2. Logical Level	Uses formal logic to represent facts.	$\forall x$ (Dog(x) → Animal(x))
3. Implementation Level	How knowledge is stored in the system.	Using rules, frames, semantic networks, etc.

Approaches / Methods of Knowledge Representation

Method	Representation	Example / Notes
1. Logical Representation (Propositional)	Uses propositional symbols and logical connectives (AND, OR, NOT, IF...THEN).	P: It is raining. Q: The ground is wet. Knowledge: P → Q
2. First Order Logic (Predicate Logic)	Uses objects, predicates, functions and quantifiers ($\forall$, $\exists$).	$\forall x$ (Human(x) → Mortal(x)) Human(Socrates) ⇒ Mortal(Socrates)
3. Semantic Networks	Graph of nodes (concepts) and edges (relations).	
4. Frames	Structure for representing stereotyped situations.	Frame: Car Slots: wheels = 4 engine = yes movesOn = road
5. Production Rules	IF-THEN rules to represent knowledge.	IF it is raining THEN take umbrella.
6. Ontologies	Formal specification of concepts, relations and constraints in a domain.	Defines class hierarchy, properties, relations. (Used in Semantic Web)

Examples

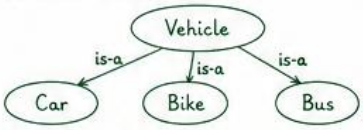
1. Propositional Logic

   P → Q, P, ∴ Q (Modus Ponens)

2. First Order Logic

   $\forall x$ (Student(x) → Person(x))
   Student(Amit)
   ∴ Person(Amit)

3. Semantic Network



4. Production Rule

   IF fever AND cough
   THEN flu.

5. Frame

   Frame: Teacher
   Slots: name = Mr. Sharma
   subject = AI
   worksIn = College

Comparison of KR Methods

Method	Expressiveness	Reasoning Support	Efficiency	Best Used When
Propositional Logic	Low	Limited	High	Simple statements, basic problems
First Order Logic	High	High	Moderate	Complex domains, precise reasoning
Semantic Networks	Moderate	Moderate	High	Concepts and relationships, hierarchies
Frames	Moderate	High (inheritance)	High	Stereotyped objects, common sense knowledge
Production Rules	High	High (forward/backward chaining)	Moderate	Expert systems, decision making
Ontologies	Very High	High	Moderate	Large scale domains, knowledge sharing

Notes

- ✓ Choice of representation depends on the problem and domain.
- ✓ More expressive representation may increase reasoning cost.
- ✓ Ontologies are widely used in AI, IoT, Semantic Web.
- ✓ KR is the foundation for reasoning, planning and learning.

Where is it Used?

- Expert Systems
- Natural Language Understanding
- Robotics and Planning
- Semantic Web and Knowledge Graphs

8. LOGIC & INFERENCE

codedsoul_05

What is Logic & Inference?

Logic is the study of valid reasoning.
Inference is the process of deriving new information (conclusions) from given knowledge (premises).

Key Idea :

If the premises are true and the inference is valid, then the conclusion must be true.

Why is it Important?

- ✓ Foundation of all reasoning in AI.
- ✓ Used in knowledge representation, planning, and problem solving.
- ✓ Helps in automated theorem proving, expert systems, and decision making.

Propositional Logic (PL)

Involves simple statements (propositions) that are either True (T) or False (F).

Basic Connectives

Symbol	Name	Meaning
$\neg P$	NOT	Negation. True if P is False.
$P \wedge Q$	AND	True if both P and Q are True.
$P \vee Q$	OR	True if at least one of P, Q is True.
$P \rightarrow Q$	IMPLIES	False only when P is True and Q is False.
$P \leftrightarrow Q$	IFF	True when P and Q have same truth value.

Truth Table Example :

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

Laws of Logic

- Identity : $P \wedge T = P$, $P \vee F = P$
- Domination : $P \vee T = T$, $P \wedge F = F$
- Idempotent : $P \vee P = P$, $P \wedge P = P$
- Double Negation : $\neg(\neg P) = P$
- De Morgan's Laws :
  $\neg(P \wedge Q) = \neg P \vee \neg Q$
  $\neg(P \vee Q) = \neg P \wedge \neg Q$

Predicate (First Order) Logic (FOL)

Extends propositional logic by using predicates, variables, functions and quantifiers.

Components

- Predicates : P(x), Human(x)
- Constants : a, John, 10
- Variables : x, y, z
- Functions : Father(x), Age(x)
- Quantifiers : $\forall$ (For all), $\exists$ (There exists)

Examples

- $\forall x$ (Human(x) $\rightarrow$ Mortal(x)) (All humans are mortal)
- $\exists x$ (Student(x) $\wedge$ Smart(x)) (There exists a student who is smart)
- $\neg\exists x$ (Bird(x) $\wedge$ CanSwim(x)) (No bird can swim)

Validity & Consistency

Term	Meaning
Valid Argument	Conclusion must be true if all premises are true.
Sound Argument	Valid argument with all true premises.
Satisfiable	At least one interpretation makes the expression true.
Tautology	True in all possible interpretations.
Contradiction	False in all possible interpretations.
Consistent KB	KB does not contain any contradiction.

Inference Rules (Rules of Inference)

Rule	Form	Example
Modus Ponens (MP)	$P,\ P \rightarrow Q \ \therefore Q$	It is raining. If it rains, the ground is wet. ∴ The ground is wet.
Modus Tollens (MT)	$\neg Q,\ P \rightarrow Q \ \therefore \neg P$	The ground is not wet. If it rains, the ground is wet. ∴ It is not raining.
Hypothetical Syllogism (HS)	$P \rightarrow Q,\ Q \rightarrow R \ \therefore P \rightarrow R$	If it rains, ground is wet. If ground is wet, roads are slippery. ∴ If it rains, roads are slippery.
Disjunctive Syllogism (DS)	$P \vee Q,\ \neg P \ \therefore Q$	Either A or B is true. A is false. ∴ B is true.
Addition	$P \ \therefore P \vee Q$	It is sunny. ∴ It is sunny or it is raining.
Simplification	$P \wedge Q \ \therefore P$	It is sunny and warm. ∴ It is sunny.
Conjunction	$P,\ Q \ \therefore P \wedge Q$	It is sunny. It is warm. ∴ It is sunny and warm.
Resolution	$P \vee Q,\ \neg P \vee R \ \therefore Q \vee R$	Clause1: P or Q Clause2: Not P or R ∴ Q or R

Inference in FOL

- ✓ Universal Instantiation (UI) : $\forall x\ P(x) \Rightarrow P(a)$
- ✓ Universal Generalization (UG) : $P(x)$ for any $x \Rightarrow \forall x\ P(x)$
- ✓ Existential Instantiation (EI) : $\exists x\ P(x) \Rightarrow P(c)$ (c is a new constant)
- ✓ Existential Generalization (EG) : $P(c) \Rightarrow \exists x\ P(x)$
- ✓ Modus Ponens, Modus Tollens etc. can also be used in FOL.

Example (Using FOL Rules)

Premises:
1. $\forall x$ (Human(x) $\rightarrow$ Mortal(x))
2. Human(Socrates)

From 1 by UI: Human(Socrates) $\rightarrow$ Mortal(Socrates)
From 2 and above by MP: Mortal(Socrates)

Common Inference Problems (Examples)

1. $\dfrac{P \rightarrow Q,\ P}{\therefore Q}$ (Modus Ponens)
2. $\dfrac{P \rightarrow Q,\ \neg Q}{\therefore \neg P}$ (Modus Tollens)
3. $\dfrac{P \vee Q,\ \neg P}{\therefore Q}$ (Disjunctive Syllogism)
4. $\dfrac{P \rightarrow Q,\ Q \rightarrow R}{\therefore P \rightarrow R}$ (Hypothetical Syllogism)
5. $\dfrac{P \wedge Q}{\therefore P}$ (Simplification)

Summary Comparison

Aspect	Propositional Logic (PL)	First Order Logic (FOL)
Basic Unit	Propositions (P, Q, R)	Predicates, Variables, Functions, Quantifiers
Expressiveness	Low	High
Representation	Facts about whole world	Facts about objects and their relations
Reasoning	Truth tables, inference rules	Unification, resolution, quantifier rules
Use Cases	Simple problems, circuits, configurations	Real world problems, AI, databases, NLP
Decidability	Decidable	Semi-decidable

Notes

- ✓ Inference derives new facts from known facts.
- ✓ Validity depends on structure, not on content.
- ✓ Soundness = Validity + True premises.
- ✓ FOL is more powerful but computationally more complex.

9. PROBABILITY & BAYESIAN REASONING

codedsoul_05

What is Probability?

Probability quantifies uncertainty.
It tells us how likely an event is to occur.

Basic Rules

- ✓ $0 \le P(A) \le 1$
- ✓ $P(S) = 1$ (S = Sample Space)
- ✓ $P(\emptyset) = 0$
- ✓ $P(A^c) = 1 - P(A)$ (Complement Rule)
- ✓ If A and B are mutually exclusive, $P(A \cup B) = P(A) + P(B)$
- ✓ If A and B are independent, $P(A \cap B) = P(A) \cdot P(B)$

Key Idea :

Bayesian reasoning updates our beliefs (probabilities) when new evidence is observed.

Prior belief + Evidence ---→ Posterior belief

Why is it Important?

- ✓ Model uncertainty in real world
- ✓ Update beliefs with new evidence
- ✓ Foundation of many AI algorithms
- ✓ Used in diagnosis, NLP, robotics, recommendation systems, etc.

Types of Probability

Type	Description	Example
Classical Probability	All outcomes are equally likely.	Rolling a fair dice
Empirical Probability	Based on observation / experiment.	% of heads in 100 coin tosses
Subjective Probability	Based on personal belief.	I think it will rain (P=0.7)
Conditional Probability	Probability of A given B is true.	$P(A \mid B)$
Joint Probability	Probability of A and B together.	$P(A \cap B)$

Conditional Probability

Probability of A given that B is true.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

Bayes' Theorem

Updates prior belief P(H) using evidence E to get posterior $P(H \mid E)$.

$$P(H \mid E) = \frac{P(E \mid H) \cdot P(H)}{P(E)}$$

- Posterior — How likely is E given H
- Prior — Initial belief about H
- Prior — Updated belief after E
- Evidence — Probability of E (Normalization)

Where, $P(E) = \sum_i P(E \mid H_i) \cdot P(H_i)$ (Law of Total Probability)

Law of Total Probability

Used to compute probability of evidence E over all possible hypotheses.

$$P(E) = \sum_i P(E \mid H_i) \cdot P(H_i)$$

Where $H_1, H_2, \ldots, H_n$ are mutually exclusive and exhaustive.

Example : Medical Test

A disease D affects 1% of the population.
Test is 99% accurate if disease is present (True Positive Rate).
False Positive Rate (when disease is absent) = 5%.
If a person tests Positive, what is the probability that he actually has the disease?

Let,
H : Person has disease
E : Test is Positive

Given,
$P(H) = 0.01$
$P(E \mid H) = 0.99$
$P(E \mid H^c) = 0.05$
$P(H^c) = 0.99$

Step 1: $P(E)$

$$P(E) = P(E \mid H)P(H) + P(E \mid H^c)P(H^c)$$
$$= (0.99)(0.01) + (0.05)(0.99)$$
$$= 0.0099 + 0.0495$$
$$= 0.0594$$

Step 2: $P(H \mid E)$

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)} = \frac{0.99 \times 0.01}{0.0594}$$
$$= 0.1667 \ (\approx 16.67\%)$$

Even after a positive test, only ~16.67% chance that the person actually has the disease!

Independence

Two events A and B are independent if

$$P(A \cap B) = P(A) \cdot P(B)$$

Equivalently,
$P(A \mid B) = P(A)$ and $P(B \mid A) = P(B)$

Note :

If A and B are not independent, knowing one affects the probability of the other.

Naive Bayes Assumption

Assumes features are conditionally independent given the class.

Class C, Features $X_1, X_2, \ldots, X_n$

$$P(C \mid X_1, \ldots, X_n) \propto P(C) \prod_{i=1}^{n} P(X_i \mid C)$$

Used in spam filtering, text classification, etc.

Comparison of Probability Concepts

Concept	Formula	Use	Example
Complement	$P(A^c) = 1 - P(A)$	Probability that A does NOT occur	Not raining if P(rain) = 0.3 → 0.7
Union (General)	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$	Probability of A or B or both	At least one of A or B
Conditional	$P(A \mid B) = P(A \cap B)\ P(B)$	Probability of A given B	P(disease | positive test)
Bayes' Theorem	$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)}$	Update belief with evidence	Medical diagnosis, spam detection
Total Probability	$P(E) = \sum_i P(E \mid H_i)\ P(H_i)$	Compute evidence probability	Sum over all causes
Independence	$P(A \cap B) = P(A)P(B)$	Events do not affect each other	Rolling two independent dice

Notes

- ✓ Bayesian reasoning mimics human learning.
- ✓ Prior can be uniform if no knowledge is available.
- ✓ With more evidence, posterior becomes more accurate.
- ✓ Beware of base rate fallacy (ignored prior probabilities).
- ✓ Naive Bayes works surprisingly well even if independence assumption is not fully true.

Applications

- Medical Diagnosis
- Spam Filtering
- Recommendation Systems
- Fraud Detection
- Robotics (Localization, Mapping)
- Natural Language Processing

10. MACHINE LEARNING FUNDAMENTALS

codedsoul_05

What is Machine Learning?

Machine Learning (ML) is a branch of AI that allows systems to learn patterns from data and make predictions or decisions without being explicitly programmed.

Key Idea :

Learn from data → Find patterns → Make predictions / decisions → Improve with more data

Why is it Important?

- ✓ Learns from data, improves over time
- ✓ Handles complex & large-scale problems
- ✓ Used in real-world intelligent applications
- ✓ Foundation for Deep Learning & AI systems

Types of Machine Learning

Type	Description	Example
1. Supervised Learning	Learn from labeled data (input → output).	Email spam classification
2. Unsupervised Learning	Find hidden patterns in unlabeled data.	Customer segmentation
3. Semi-Supervised Learning	Use both labeled & unlabeled data.	Medical image analysis
4. Reinforcement Learning	Learn by interacting with environment and getting rewards.	Game playing, Robotics

ML Workflow

1. Problem Definition — Define goal & success criteria
2. Data Collection — Gather relevant data
3. Data Preprocessing — Clean data, handle missing values, normalize, encode, etc.
4. Feature Engineering — Select / create important features
5. Model Training — Train algorithm on training data
6. Evaluation — Measure performance on test data
7. Deployment — Deploy model & monitor performance

Common ML Algorithms

Category	Algorithms	Used For
Classification	Logistic Regression, Decision Tree, Random Forest, SVM, Naive Bayes, KNN	Predicting discrete classes (Yes/No, Cat/Dog)
Regression	Linear Regression, Ridge, Lasso, SVR, Decision Tree Regressor	Predicting continuous values (Price, Salary)
Clustering	K-Means, Hierarchical, DBSCAN	Grouping similar data points
Dimensionality Reduction	PCA, t-SNE	Reducing features while keeping important info
Ensemble Methods	Random Forest, Gradient Boosting, XGBoost, AdaBoost	Improved accuracy by combining models

Train - Test Split

Divide data into training and testing sets.



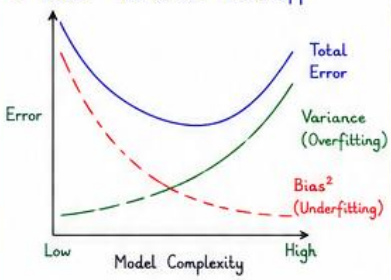
- ✓ Training set → Used to train the model
- ✓ Test set → Used to evaluate performance on unseen data
- ✓ Helps measure how well the model generalizes to real world

Evaluation Metrics

Metric	Used For	Formula / Explanation
Accuracy	Classification	(TP + TN) / (TP + TN + FP + FN)
Precision	Classification	TP / (TP + FP)
Recall (Sensitivity)	Classification	TP / (TP + FN)
F1-Score	Classification	2 × (Precision × Recall) / (Precision + Recall)
MAE	Regression	Mean(| y_true - y_pred |)
MSE	Regression	Mean((y_true - y_pred)²)
RMSE	Regression	√MSE

Where, TP: True Positive, TN: True Negative,
FP: False Positive, FN: False Negative

Bias - Variance Tradeoff



- ✓ Low complexity → High bias, Low variance (underfitting)
- ✓ High complexity → Low bias, High variance (overfitting)
- ✓ Best model is the sweet spot in the middle.

Overfitting vs Underfitting

Underfitting (High Bias)

- Model too simple
- Cannot capture patterns
- High training error
- High test error
- Solution: Increase model complexity, add features



Overfitting (High Variance)

- Model too complex
- Captures noise as well
- Low training error
- High test error
- Solution: Reduce complexity, regularization, more data



Common Data Types

Data Type	Description	Example
Numerical	Numbers (continuous or discrete)	Age, Height, Income
Categorical (Nominal)	Categories with no order	Color (Red, Blue), City (Delhi, Mumbai)
Ordinal	Categories with order	Small < Medium < Large, Ratings (1,2,3,4,5)
Binary	Two categories (0 or 1)	Yes/No, True/False, Spam/Not Spam

Summary Table

Concept	Key Point	Remember This
Supervised Learning	Learn from labeled data	We know the answer
Unsupervised Learning	Find patterns in unlabeled data	We don't know the answer
Model Evaluation	Use metrics on test data	Don't evaluate on training data
Overfitting	Too complex → learns noise	High train accuracy, low test accuracy
Underfitting	Too simple → misses patterns	Low train accuracy, low test accuracy
Generalization	Perform well on unseen data	Goal of all ML models!

Real-World Applications

- Image Recognition (Face, Object Detection)
- Recommendation Systems (Amazon, Netflix)
- Spam Detection (Email Filtering)
- Fraud Detection (Banking, Finance)
- Autonomous Vehicles (Self-driving Cars)
- Chatbots & Virtual Assistants

11. SUPERVISED & UNSUPERVISED LEARNING

codedsoul_05

What is the Difference?

Supervised Learning : Learn from labeled data (input + output).
Goal is to predict the output for new input.

Unsupervised Learning : Learn from unlabeled data (only input).
Goal is to find hidden patterns or structure.

Key Idea

Supervised → Predict / Classify
Unsupervised → Discover / Group

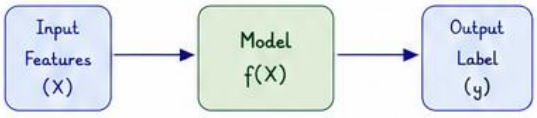
Learn with answers vs Learn without answers

Why is it Important?

- ✓ Supervised is used for prediction problems.
- ✓ Unsupervised is used for pattern discovery.
- ✓ Both are widely used in real-world AI applications.

SUPERVISED LEARNING

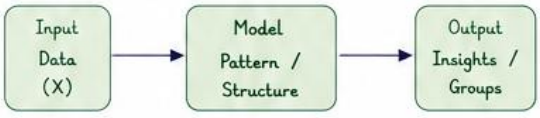
Uses labeled dataset (X, y). Model learns mapping from input (X) to output (y).



Examples : Email spam classification, House price prediction, Image classification.

UNSUPERVISED LEARNING

Uses unlabeled dataset (only X). Model finds patterns, groups or structure in data.



Examples : Customer segmentation, Market basket analysis, Anomaly detection.

Common Supervised Learning Algorithms

Algorithm	Type	Used For	Example
Linear Regression	Regression	Predict continuous values	House price, Salary
Logistic Regression	Classification	Binary classification	Spam / Not Spam
Decision Tree	Both	Classification & Regression	Loan approval, Customer churn
Random Forest	Both	Improve accuracy, reduce overfitting	Fraud detection
Support Vector Machine (SVM)	Classification	High dimensional classification	Image, Text classification
k-Nearest Neighbors (kNN)	Both	Simple, instance based learning	Recommender systems
Gradient Boosting (XGBoost)	Both	High performance on structured data	Tabular data predictions

Common Unsupervised Learning Algorithms

Algorithm	Used For	Example
K-Means Clustering	Clustering (grouping similar data)	Customer segmentation
Hierarchical Clustering	Clustering with tree structure	Document clustering
DBSCAN	Density based clustering	Anomaly detection
PCA (Principal Component Analysis)	Dimensionality Reduction	Reduce features, visualization
t-SNE	Dimensionality Reduction (for visualization)	Plot high dimensional data in 2D/3D
Apriori Algorithm	Association Rule Mining	Market basket analysis
Autoencoders	Representation Learning (Neural Network based)	Anomaly detection, feature learning

Dataset Example

Supervised (Labeled Data)

Size (sq.ft)	Bedrooms	Price (in $)
1200	2	120000
1500	3	150000
1800	3	180000
2000	4	200000
...	...	...

Model learns: (Size, Bedrooms) → Price

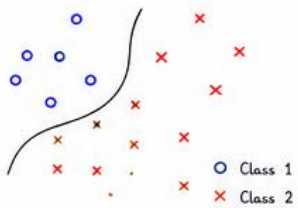
Unsupervised (Unlabeled Data)

Size (sq.ft)	Bedrooms
1200	2
1500	3
1800	3
2000	4
...	...

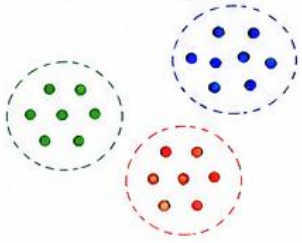
Model finds groups / patterns

Visualization

Supervised (Classification)
(Finds decision boundary)



Unsupervised (Clustering)
(Finds natural groups)



Comparison

Aspect	Supervised Learning	Unsupervised Learning
Data	Labeled (X, y)	Unlabeled (X only)
Goal	Predict output / label	Find patterns / structure
Examples	Regression, Classification	Clustering, Association, Dimensionality Reduction
Algorithms	Linear Regression, SVM, Random Forest, etc.	K-Means, PCA, DBSCAN, Apriori, etc.
Evaluation	Accuracy, F1-Score, RMSE (needs ground truth)	Silhouette Score, Inertia, Visualization (no labels)
Use Cases	Prediction, Forecasting, Classification	Segmentation, Discovery, Pattern Analysis

When to Use Which?

Use Supervised When:

- ✓ You have labeled data
- ✓ You know the target/output
- ✓ Goal is prediction or classification
- ✓ Example: Predict house price

Use Unsupervised When:

- ✓ You have unlabeled data
- ✓ You don't know the output
- ✓ Goal is to discover patterns
- ✓ Example: Group customers

Popular Real-World Applications

- Spam Detection (Supervised)
- House Price Prediction (Supervised)
- Disease Prediction (Supervised)
- Product Recommendation (Both)
- Customer Segmentation (Unsupervised)
- Market Basket Analysis (Unsupervised)
- Anomaly Detection (Unsupervised)
- Image Classification (Supervised)

Notes

- ✓ Real-world problems often use both together.
- ✓ First use unsupervised to understand data & segment.
- ✓ Then use supervised models to make accurate predictions.
- ✓ Quality of data > Choice of algorithm.

12. NEURAL NETWORKS & DEEP LEARNING

codedsoul_05

What is Deep Learning?

Deep Learning (DL) is a subset of Machine Learning that uses Artificial Neural Networks (ANNs) with multiple layers to learn complex patterns from large amounts of data.

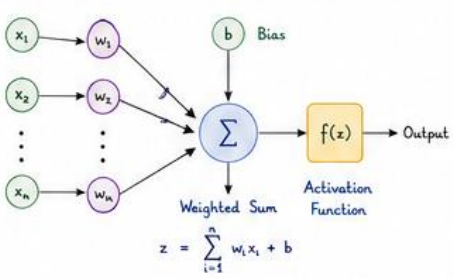
Key Idea

Neural Networks → Learn representations
Deep Learning → Learn hierarchical representations automatically

Why is it Important?

- ✓ Learns complex patterns automatically
- ✓ Works well with unstructured data (images, text, audio)
- ✓ State-of-the-art in many real-world tasks
- ✓ Powering modern AI applications

1. Artificial Neuron (Perceptron)

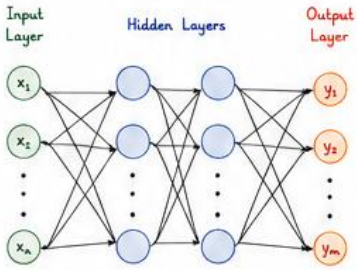


$$z = \sum_{i=1}^{n} w_i x_i + b$$

Output: $y = f(z)$

The neuron learns weights (w_i) and bias (b) to minimize error.

2. Structure of a Neural Network



Information flows from input → hidden layers → output.

More hidden layers = Higher representation learning

3. Activation Functions

Function	Formula	Graph	Range	Used In
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$		(0,1)	Output layer (Binary Classification)
Tanh	$\tanh(z)$		(-1,1)	Hidden layers
ReLU	$f(z) = \max(0,z)$		[0, ∞)	Hidden layers (Default)
Leaky ReLU	$f(z) = \max(\alpha z, z)$		(-∞, ∞)	Hidden layers (avoids dying ReLU)
Softmax	$\sigma(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$		(0,1) $\Sigma = 1$	Output layer (Multi-class Classification)

4. Types of Neural Networks

Type	Description	Example Use
Feedforward Neural Network (FNN)	Connections move in one direction only (input → output)	Tabular data, basic prediction tasks
Convolutional Neural Network (CNN)	Uses convolution layers to capture spatial patterns	Image classification, object detection
Recurrent Neural Network (RNN)	Has loops to handle sequential data	Text, time series, speech
Long Short-Term Memory (LSTM)	Special RNN that remembers long-term dependencies	Language modeling, time series
Transformer	Uses self-attention mechanism	NLP (BERT, GPT), Vision Transformers

5. Forward Propagation

1. Input data is passed to the network
2. Each neuron computes weighted sum
3. Activation function is applied
4. Output is produced

For layer l:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = f(Z^{[l]})$$

$A^{[0]}$ = Input features (X)

6. Loss Functions (Cost)

- Regression : $MSE = \frac{1}{n} \sum_{n}^{n} (\hat{y}_i - y_i)^2$
- Binary Classification:

$$BCE = -\frac{1}{n} \sum_{n}^{n} \left[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \right]$$

- Multi-class Classification:
  Categorical Cross Entropy

7. Backpropagation (Learning)

1. Calculate loss
2. Compute gradient of loss w.r.t each weight
3. Update weights using Gradient Descent

$$w = w - \eta \frac{\partial L}{\partial w}$$

Where, η = Learning Rate

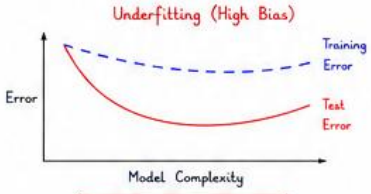
Training Loop

```
For epoch in range(N):
    For each batch in data:
        Forward Propagation
        Compute Loss
        Backpropagation
        Update Weights
```

8. Key Hyperparameters

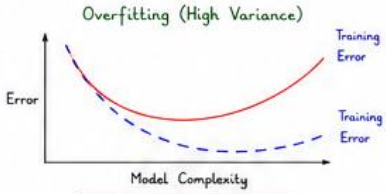
Hyperparameter	Description
Learning Rate (η)	Step size while updating weights
Batch Size	Number of training samples per update
Epochs	Number of times entire dataset is seen
Optimizer	Algorithm to update weights (SGD, Adam, RMSProp)
Dropout Rate	Randomly drops neurons to prevent overfitting
Number of Layers	Depth of the network
Activation Function	Adds non-linearity (ReLU, Tanh, etc.)

Underfitting (High Bias)



Too simple model, can't capture patterns

Overfitting (High Variance)



Too complex model, learns noise

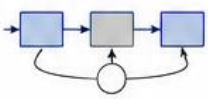
10. Popular Architectures

CNN



Convolutions capture spatial features. Used in images, video, medical imaging.

RNN / LSTM

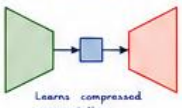


Transformer



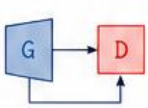
Uses self-attention instead of RNN. Used in BERT, GPT, T5, ViT, etc.

Autoencoder



Learns compressed representation. Used for anomaly detection, denoising.

GAN



Generator creates fake data, Discriminator separates real vs fake. Used in image generation.

11. Deep Learning Frameworks

- **PyTorch** — Dynamic computation graph, easy debugging
- **TensorFlow** — Static graph (TF 1.x) Eager execution (TF 2.x)
- **Keras** — High-level API Runs on top of TF
- **MXNet** — Scalable, flexible, enterprise ready

12. Real-World Applications

- Image Recognition (Face, Object Detection)
- Speech Recognition & Synthesis
- Natural Language Processing (Chatbots, Translation)
- Recommender Systems (Netflix, YouTube)
- Autonomous Vehicles
- Healthcare (Diagnosis, Drug Discovery)

Notes

- ✓ Deep Learning needs large data + high computation.
- ✓ Feature engineering is learned automatically.
- ✓ More data + more compute → better performance.
- ✓ Start simple, then increase complexity.

Remember

Data → Model → Train → Evaluate → Improve
This is the core cycle of Deep Learning!

2. Quick Formula Sheet

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \qquad A^{[l]} = f(Z^{[l]})$$

$$Loss = L(y, \hat{y})$$

$$Update: \; w = w - \eta \frac{\partial L}{\partial w}, \quad b = b - \eta \frac{\partial L}{\partial b}$$

13. NLP & COMPUTER VISION

codedsoul_05

- **What are they?**

NLP (Natural Language Processing) enables computers to understand, interpret and generate human language.

Computer Vision enables computers to understand and interpret visual information from images and videos.

Key Idea

NLP → Understand & generate human language
CV → Understand & interpret visual world

Both convert real-world data into meaningful information.

Why are they Important?

- ✓ Essential for many AI applications
- ✓ Huge real-world impact (search, health, cars, robots, chatbots)
- ✓ Growing research and industry demand
- ✓ Used together in multimodal AI systems

A. NLP (NATURAL LANGUAGE PROCESSING)

Goal: Enable machines to understand, interpret and generate human language.

1. NLP Pipeline

Text Input (Raw text) → Preprocessing (Clean, tokenize, remove stopwords) → Representation (Convert to numbers) → Model (Train / Infer) → Output (Answer / Label / Generated Text)

2. Common NLP Tasks

Task	Description	Examples
Text Classification	Assign text to predefined categories	Spam detection
Sentiment Analysis	Determine sentiment (pos/neg/neutral)	Review analysis
Named Entity Recognition	Identify entities (Person, Org, Location, Date, etc.)	Extract names
Machine Translation	Translate text from one language to another	EN → HI
Question Answering	Answer questions from given text	Chatbots
Text Summarization	Shorten long text while keeping important info	News summary
Language Generation	Generate human-like text	AI writing, ChatGPT

3. Text Representation

- Bag of Words (BoW) / TF-IDF
- Word Embeddings: Word2Vec, GloVe
- Contextual Embeddings: BERT, RoBERTa, DistilBERT, etc.

Sentence: "AI is transforming the world." → Tokenization (AI, is, transforming, the, world, .) → Embedding [0.12, -0.09, ...] [0.45, 0.31, ...] [(... ...)]

4. Popular Models / Architectures

Model	Use
RNN / LSTM / GRU	Sequence modeling, language tasks
Transformer	Foundation for modern NLP models
BERT	Understanding tasks (classification, NER, QA)
GPT (Generative Pretrained Transformer)	Text generation, chatbots, content creation
T5 / BART	Seq2Seq tasks (translation, summarization)

5. NLP Applications

- Chatbots & Virtual Assistants
- Machine Translation
- Search & Information Retrieval
- Sentiment Analysis (Social Media)
- Text Summarization
- Spam Detection
- Question Answering
- Content Generation

B. COMPUTER VISION (CV)

Goal: Enable machines to understand and interpret visual information.

1. CV Pipeline



2. Common CV Tasks

Task	Description	Examples
Image Classification	Classify entire image into a class	Cat / Dog / Car
Object Detection	Detect & locate objects in image	YOLO, SSD, Faster R-CNN
Image Segmentation	Pixel-wise classification	Medical imaging
Semantic Segmentation	Classify each pixel into a class	Road, Building, Tree
Instance Segmentation	Detect each object instance separately	Detect all people
Facial Recognition	Identify or verify a person	Face ID, Attendance
Object Tracking	Track object across frames	Surveillance, Sports

3. Popular Architectures (CV)

CNN (Convolutional Neural Network)	Classic model for image tasks
ResNet	Uses skip connections to train deep models
Inception / GoogLeNet	Multi-scale feature extraction
EfficientNet	High accuracy with fewer parameters
YOLO (You Only Look Once)	Real-time object detection
U-Net	Popular for segmentation (medical, etc.)
Vision Transformer (ViT)	Transformer for images

4. CV Applications

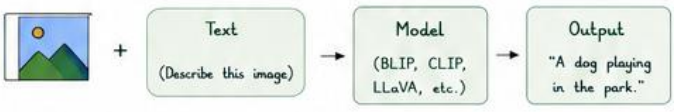
- Image Classification
- Object Detection
- Autonomous Vehicles
- Medical Image Analysis
- Surveillance & Security
- Retail (Shelf, Product Detection)
- Industrial Inspection
- OCR (Text in Images)

5. NLP vs CV at a Glance

Aspect	NLP	Computer Vision
Data Type	Text / Speech	Images / Videos
Input Example	"Hello, how are you?"	(Photo of a cat)
Core Techniques	Tokenization, Embeddings, Transformers	CNNs, Feature Maps, Transformers (ViT)
Output Example	Sentiment: Positive	Label: Cat (99%)
Challenges	Ambiguity, Context, Sarcasm	Lighting, Occlusion, Viewpoint
Popular Models	BERT, GPT, T5	ResNet, YOLO, U-Net, ViT

6. Together: Multimodal AI

NLP + CV = Understand both text and visuals.



Key Takeaways

- ✓ NLP helps machines understand human language.
- ✓ CV helps machines understand the visual world.
- ✓ Both are core to many real-world AI applications.
- ✓ Multimodal models combine NLP + CV for powerful results.

Examples You Use Daily

- Google Search (NLP+ CV)
- Google Lens (CV)
- ChatGPT (NLP)
- Self-driving Cars (CV)
- Voice Assistants (NLP)
- Instagram Filters (CV)

Remember

NLP = Understanding words
CV = Understanding visuals
Together = Understanding the world!

14. GENERATIVE AI, LLMs & RAG

codedsoul_05

What is Generative AI?

Generative AI creates new content (text, images, code, audio, video) based on patterns learned from large amounts of data.

Examples: ChatGPT, DALL-E, GitHub Copilot, Midjourney, Claude.

Key Idea

Models learn from data → understand patterns → generate meaningful, human-like outputs.
LLMs are a type of Generative AI focused on text.
RAG combines LLMs + external knowledge for accurate, up-to-date answers.

☆ Why is it Important?

- ✓ Automates content creation & coding
- ✓ Enhances productivity & decision making
- ✓ Understands & generates human language
- ✓ Connects AI with real-world knowledge (RAG)
- ✓ Powering chatbots, copilots, search, agents & more

1. Generative AI - Types

Text Generation	Image Generation	Code Generation	Other Modalities
Articles, stories, emails, code, summaries Ex: ChatGPT	Create images from text prompts Ex: DALL·E, Midjourney	Generate & auto-complete code Ex: GitHub Copilot	Audio, Video, 3D, Music, etc. Ex: Sora, Runway

2. LLMs (Large Language Models)

- Trained on massive text data to understand and generate human language.
- Examples: GPT-4o, Claude, Gemini, Llama 3, Mistral, PaLM.

How LLMs Work (Simplified)

Input Text (Tokenized) → Embedding (Meaning) → Transformer (Attention) → Next Token Prediction → Generated Text

Common Terms

Term	Meaning
Token	Small piece of text (word / part of word / punctuation)
Context Window	How much text the model can consider at once (e.g., 8k, 32k, 128k tokens)
Parameters	Internal values model learns during training (in billions)
Temperature	Controls randomness. Low → deterministic, High → creative
Top-k / Top-p	Limits next token choices to improve quality & relevance
Fine-tuning	Further training on specific data for better performance
RLHF	Reinforcement Learning from Human Feedback

3. LLM Capabilities

- Chat & Q&A
- Summarization
- Translation
- Content Generation
- Code Generation
- Reasoning & Problem Solving
- Data Extraction
- Tool / Function Calling (Agents)

4. Limitations of LLMs

- ✗ May generate incorrect / hallucinated information
- ✗ Knowledge can be outdated
- ✗ No direct access to private or domain-specific data
- ✗ Cannot verify facts by itself
- ✗ May lack explainability and reasoning in complex tasks

Mitigation

- ✓ Use better prompts
- ✓ Verify from trusted sources
- ✓ Use RAG for factual accuracy
- ✓ Human-in-the-loop
- ✓ Fine-tune / customize

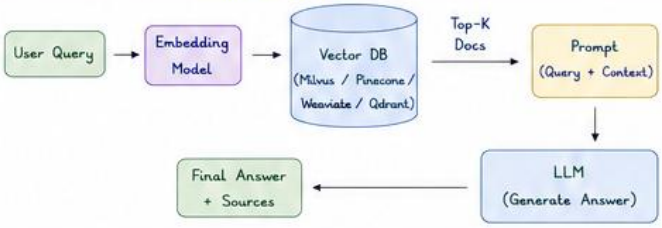
5. What is RAG?

RAG (Retrieval Augmented Generation) combines retrieval of relevant documents with LLM generation to provide accurate, context-aware and up-to-date answers.

RAG Workflow



RAG Architecture



6. RAG Components

Component	Description
Data Ingestion	Load data from files/web/DB/APIs
Chunking	Split data into small chunks (e.g., 200–1000 tokens)
Embedding	Convert text chunks to vectors using embedding model
Vector Database	Store vectors for semantic search
Retriever	Find top-k relevant chunks for the query
Augment	Add retrieved context to the prompt
LLM	Generate answer using augmented prompt
Output	Answer with sources / references

7. RAG vs Fine-tuning

Aspect	RAG	Fine-tuning
Data	Uses external data at inference time	Requires training on domain data
Knowledge Freshness	Up-to-date (if data updated)	Static (until re-trained)
Cost	Lower (no re-training)	High (training GPU cost)
Customization	Flexible (change data easily)	Less flexible
Accuracy (Factual)	Higher (uses real documents)	May hallucinate
Use Cases	Q&A, Support bots, Docs search	Style transfer, Classification, Domain adaptation

8. Popular Models

Proprietary Models

- OpenAI GPT-4o / GPT-4
- Anthropic Claude 3
- Google Gemini 1.5
- Microsoft Copilot (GPT-4 based)

Open Source Models

- Llama 3 (Meta)
- Mistral 7B / Mixtral 8x22B
- Gemma (Google)
- Phi-3 (Microsoft)
- Qwen (Alibaba)

9. RAG Use Cases

- Enterprise Knowledge Search
- Customer Support Chatbots
- Research & Technical Q&A
- Legal / Compliance Assistant
- Healthcare Information Retrieval
- Product / Document Search
- Education & Tutoring Bots
- Code / API Documentation Bots

10. Best Practices

- ☆ Use high-quality, clean, and structured data
- ☆ Choose the right chunk size (not too small or large)
- ☆ Use strong embedding models (e.g., text-embedding-3-large)
- ☆ Tune Top-k for better context retrieval
- ☆ Show sources / citations for transparency
- ☆ Monitor, evaluate and keep data fresh

11. RAG Evaluation Metrics

- ☆ Context Relevance (Retriever Recall@k)
- ☆ Answer Faithfulness (Grounded in context)
- ☆ Answer Accuracy
- ☆ Helpfulness / User Satisfaction
- ☆ Latency / Cost

12. Tools & Frameworks

- LangChain
- LlamaIndex
- Haystack
- Vector DBs: Pinecone, Milvus, Weaviate, Qdrant
- Models API: OpenAI, Anthropic, Gemini
- Embeddings: OpenAI, Cohere, HuggingFace
- Orchestration: LangGraph, CrewAI
- Deployment: Docker, FastAPI, Streamlit

Remember

LLMs are powerful but may not know everything.
RAG helps LLMs look up the right information before answering.



Together, we build AI that is Smarter, Reliable & Trustworthy!

15. AI INTERVIEW & REAL WORLD CONCEPTS

Master Concepts. Crack Interviews. Build Real World AI.

codedsoul_05

Why is this Important?

- ✓ Helps you crack AI/ML interviews
- ✓ Builds strong conceptual foundation
- ✓ Connects theory with real-world applications
- ✓ Shows problem solving & system thinking
- ✓ Makes you job-ready

Key Idea

- → Understand core concepts deeply
- → Practice how to explain & apply them
- → Solve real-world problems
- → Build projects & case studies
- → Stay updated & keep learning

AI Interview Strategy

- > Understand the question clearly
- > Break it down, think out loud
- > Use structured approach
- > Start simple, then optimize
- > Communicate, don't just code

1. Frequently Asked AI Interview Questions

Topic	Sample Questions
Machine Learning Basics	What is ML? Types of ML? Bias-Variance Tradeoff?
Supervised Learning	Linear Regression vs Logistic Regression? When to use which algorithm?
Unsupervised Learning	K-Means working? How to choose K? PCA vs t-SNE?
Deep Learning	Difference between CNN, RNN, LSTM? How does Backpropagation work?
NLP	What is Embedding? Word2Vec vs GloVe? How does Transformer work?
Computer Vision	How does YOLO work? Image classification pipeline?
System Design (AI)	Design a Recommendation System Design a Chatbot / Search Engine
Behavioral	Tell me about a project you built. How do you handle a new problem?

2. How to Approach Any AI Problem

Step 1: Understand
Understand the problem, inputs, outputs & constraints.

Step 2: Break Down
Break into smaller sub-problems. Identify key components.

Step 3: Plan
Think of algorithms / models / data structures.

Step 4: Solve
Start simple. Solve the core part first.

Step 5: Optimize
Handle edge cases, scale, optimize performance.

Step 6: Communicate
Explain your approach, trade-offs & assumptions clearly.

3. Real World AI Concepts (With Examples)

Concept	Real World Example
Classification	Spam detection, Disease prediction, Image classification (Cat vs Dog)
Regression	House price prediction, Stock price prediction, Sales forecasting
Clustering	Customer segmentation, Market basket analysis
Recommendation System	Netflix movie suggestions, Amazon product recommendations
Anomaly Detection	Fraud detection, Fault detection in machines, Credit card fraud
NLP	Chatbots, Language translation, Sentiment analysis
Computer Vision	Face detection, Self-driving cars, Medical image analysis
Generative AI	Text generation (ChatGPT), Image generation (DALL-E), Code generation

4. AI System Design Interview – Key Patterns

Recommendation System	User data → Candidate generation → Ranking → Re-ranking
Search Engine	Crawling → Indexing → Ranking → Serving results
Chatbot / LLM System	User Query → Retrieve Context → LLM → Response
Real-time Prediction	Data Ingestion → Feature Store → Model Serving → Prediction
Computer Vision System	Image Capture → Preprocess → Model Inference → Postprocess

5. Important AI Concepts (Quick Revision)

- Overfitting vs Underfitting:
  Overfit = High variance, Underfit = High bias
- Activation Functions: ReLU, Sigmoid, Tanh
- Loss Functions: MSE, Cross Entropy
- Optimization: Gradient Descent, Adam, RMSProp
- Regularization: L1, L2, Dropout
- Evaluation Metrics: Accuracy, Precision, Recall, F1-Score, ROC-AUC
- Cross Validation: K-Fold CV
- Batch vs Stochastic vs Mini-batch GD
- Epoch, Batch Size, Learning Rate

6. Real World Project Ideas

- Email Spam Detection (NLP + ML)
- Fake News Detection (NLP)
- Movie Recommendation System
- Plant Disease Classification (CV)
- Handwritten Digit Recognizer (CV)
- Chatbot using RAG + LLM
- Stock Price Prediction (Time Series)
- Fraud Detection System

7. AI in Real World – Industry Use Cases

Industry	AI Applications
Healthcare	Disease diagnosis, Drug discovery, Medical imaging
Finance	Fraud detection, Algorithmic trading, Credit scoring
E-commerce	Recommendation, Demand forecasting, Chatbots
Automotive	Self-driving cars, Predictive maintenance
Education	Personalized learning, Automated grading
Manufacturing	Quality inspection, Predictive maintenance
Marketing	Customer segmentation, Ad targeting, Sentiment analysis

8. Explain These in Interviews

Confusion Matrix	TP, TN, FP, FN – used in classification evaluation.
Precision vs Recall	Precision = Correct / Predicted Recall = Correct / Actual
ROC Curve & AUC	Measures model's ability to distinguish classes.
Feature Engineering	Process of creating features from raw data.
Data Leakage	Train & test data information mixing.
Handling Imbalanced Data	Oversampling, Undersampling, Class weights.

9. Do's & Don'ts in Interviews

Do's ✓

- ✓ Understand & clarify the problem
- ✓ Think out loud & communicate
- ✓ Start with brute force, then optimize
- ✓ Handle edge cases
- ✓ Ask for examples if not clear
- ✓ Stay calm & confident

Don'ts ✗

- ✗ Don't jump to solution
- ✗ Don't stay silent
- ✗ Don't over-complicate
- ✗ Don't ignore edge cases
- ✗ Don't give up easily
- ✗ Don't memorize without understanding

10. Useful Resources

- Books: Python Crash Course, Hands-On ML, Deep Learning (Goodfellow)
- Courses: Andrew Ng (Coursera), Fast.ai, CS231n, DeepLearning.AI
- Practice: LeetCode, Kaggle, HackerRank, Papers with Code
- Stay Updated: arXiv, Medium, Towards AI, AI Twitter (X)

11. Final Checklist Before Interview

- ✓ Revise all core AI/ML concepts
- ✓ Practice coding & problem solving
- ✓ Build at least 2–3 projects
- ✓ Understand real world use cases
- ✓ Practice system design for AI products
- ✓ Mock interviews & communicate clearly

Stay consistent. Keep building. You've got this! 💪

12. One Line Concept Recap

AI	ML	Deep Learning	NLP	Computer Vision	RAG	LLM	Generative AI
Machines doing tasks that normally require human intelligence.	AI that learns from data & improves with experience.	Subset of ML using neural networks with many layers.	AI that understands & generates human language.	AI that interprets & understands images/videos.	Combines retrieval (search) + generation (LLM) for accuracy.	Large models trained on massive text data (e.g., GPT, Llama).	AI that creates new content (text, image, code, audio, etc.).

☆ ------→ Learn • Build • Solve • Repeat ------ ☆ Keep Learning. Keep Growing. Stay Curious! ☆